

connection between LoRa module and ESP32 board are made as following:

- ESP32 pin 16 to Tx of LoRa, ESP32 pin 17 to Rx of LoRa
- LoRa M1, M2 to ESP32 GND pin
- LoRa Aux to +ve pin of ESP32 (3.3V)
- LoRa GND to ESP32 GND
- LoRa -ve to +ve pin of ESP32 (3.3V)

The ESP32 gets data on its Tx2, Rx2 (17,16) via SS radio interface and, after connecting to Wi-Fi, sends it to internet servers. It is not necessary to upload the data on a public cloud only. It can also be sent to a personal or corporate server. The following site can be used to upload data to a LAMP server using ESP32/ESP8266: <https://electronicsforu.com/electronicprojects/software-projects-ideas/distributed-cloudcomputing-esps>

For uploading data to thingspeak server, one needs to have an API upload key, which may be obtained after registering with the *thingspeak.com* site. After having the API key, the sensor data is lined up in a string, which is then parsed through the following:

```
http.begin(poststr);
int httpCode=http.GET();
if (httpCode > 0) { //Check for the
returning code
String payload = http.getString();
Serial.println("Updated");
}
```

Construction and testing

We have used two ESP32s, which are connected to two different laptops, as shown in Fig. 4. To each of the ESP32 a LoRa module is connected with suitable wire connections. Softwareserial works only on Arduino, and hardware serial works only on ESP32. Hence, the module is connected on the hardware serial port. As we are validating the original design, the number “3456” was

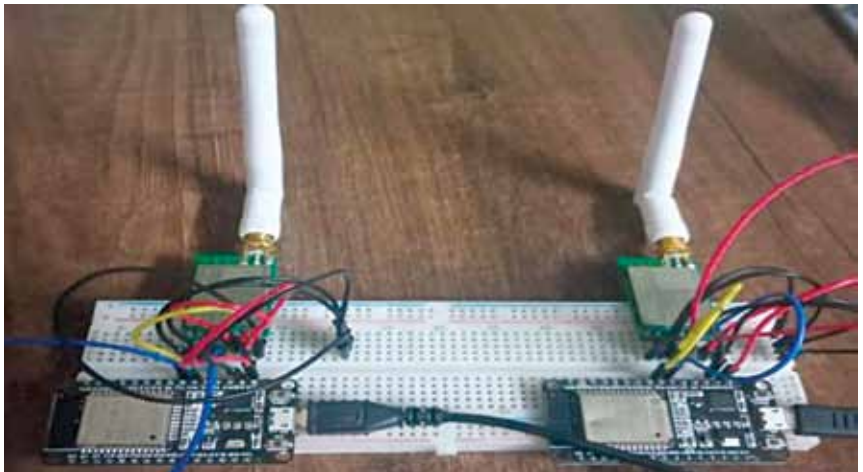


Fig. 5: Two prototype sets used during testing

given as the number to be sent from the sender. The number was initialised in the character data type. Refer the screenshot shown in Fig. 4.

So, after sending the number “3456” from the sender node, we first verify if the data is reaching the receiver. In the serial monitor of the receiver the number “3456” was being displayed, hence establishing the fact that the two modules are communicating with each other. The baud rate was set to 115200. We need to configure the serial monitor to the same baud rate in order to see the received data.

After verifying that the data is successfully being relayed to the receiver, we can use the internet to upload this received data onto the ThingSpeak server. ThingSpeak is a cloud based IoT analytics platform, which has integrated support from MATLAB. In ThingSpeak, the received data can be viewed in the CSV format as well, among the different types of compatible formats. ThingSpeak has visual tools, which helps us to see the flow of data with respect to time. It also has a numerical visual tool, which helps us to see

the number being received by the receiver and sent to ThingSpeak.

Every user on ThingSpeak can make channels for their data. Every channel has a unique channel number and an API key, and these are used as channel identifiers. The receiver node has to be connected to the internet in order to upload data onto the server. After connecting the receiver ESP32 to the internet, the code starts pushing the received data onto the server. The code uses HTTP GET to request the JSON, after which the data gets uploaded onto the server. The HTTP status code of 200 indicates that our request has been accepted.

With the help of the elaborative widgets on the ThingSpeak interface, a real-time view of the data transfer can be seen graphically. We may verify that the number “3456” is being displayed on both widgets of the ThingSpeak server, thus confirming that the data has reached the server.

The downloaded data in CSV format includes timestamps, according to one's time zone. The GMT time zone is used as default for this. The CSV files show the time-wise arrival of data on the server. **EFY**

EFY Note

The source code of this project is available

for free download at www.electronicsforu.com



Somnath Bera is an electronics and IoT enthusiastic working as General Manager at NTPC